

BCA304-5: DATA ENGINEERING

Digital Book: Units 1 & 2

Data Lifecycle & Engineering Foundations | Data Processing Using Python

Team Members:

1. Hardik: Unit 1
2. Adrino Rosario James: Unit 2

Individual Contribution Statement

As per activity guidelines, each team member has contributed independently to the content of this digital book. The breakdown of contribution is as follows:

Team Member	Unit Covered	Topic
Hardik	Unit 1	Data Lifecycle and Engineering Foundations
Adrino Rosario James	Unit 2	Data Processing Using Python

Evaluation Note: Since assessment is based entirely on individual contribution, each section below is self-contained and can be reviewed independently under the author's name.

Unit 1: Data Lifecycle and Engineering Foundations

Author: Hardik

1.1 Why This Unit Matters

Every time we use a mobile app, whether ordering food, checking a bank balance, or scrolling social media, we generate data. A single click is a fact, but a single fact on its own is not very useful. Data Engineering is the discipline that takes this raw, scattered information and turns it into something an organisation can trust and act on.

A simple example makes the stakes clear: imagine opening a banking app and seeing your balance jump from ₹5,000 to ₹50,000 overnight. Before reacting, the real question is: is the system wrong, or is the data wrong? Before any analysis or decision is made, we must be able to answer: can we trust this data?

1.2 What is Data Engineering?

Data Engineering is the development, implementation, and maintenance of the systems and processes that take in raw, messy data and convert it into high-quality, consistent, and usable information. It is the invisible infrastructure behind every dashboard, fraud alert, and recommendation a user sees.

1.3 The Data Engineering Lifecycle

The lifecycle is a structured, repeatable framework describing how data moves from creation to final use. It consists of the following stages:

- **Generation:** data is created at the source
- **Ingestion:** data is collected and moved into the processing environment
- **Storage:** data is held in a reliable, scalable system (this underpins every other stage, not just one step)

- **Transformation (Processing):** raw data is cleaned, standardised, and enriched with business logic
- **Serving (Analysis):** transformed data reaches dashboards, models, and stakeholders
- **Archival:** older data is moved to low-cost, long-term storage for compliance

These stages are supported by six constant undercurrents that run through the entire lifecycle: Security, Data Management, DataOps, Data Architecture, Orchestration, and Software Engineering.

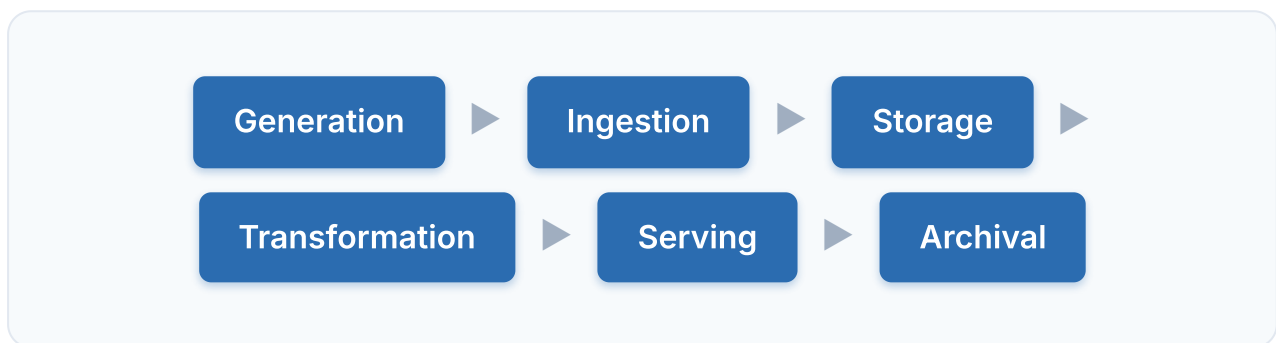


Figure 1.1: The Data Engineering Lifecycle

Stage 1: Generation

Data is created from real-world sources such as mobile apps, websites, sensors, and business systems. For example, a food delivery app generates order details, search queries, GPS location, payment information, and behavioural data (such as how long a user hovers over a menu). A bank transfer begins the same way. A customer taps "Send" on a mobile app, and this creates a transaction record inside the bank's OLTP (Online Transaction Processing) system, with a SQL database logging the exact millisecond of the request.

This is the starting point of the lifecycle, and the quality of data generated here affects every later stage: a mistake at Generation cannot be fully corrected downstream.

Stage 2: Ingestion

Ingestion is the process of collecting and moving data from source systems into a central processing environment. Without ingestion, data remains stranded in isolated files or databases and cannot be cleaned, validated, or analysed. There are two broad approaches:

- **Batch processing:** data is collected at fixed intervals (hourly, daily)

- **Real-time (streaming) processing:** data is captured and moved almost instantly

In the banking example, a technique called CDC (Change Data Capture) is used: the bank "watches" database logs, and every time a balance changes, that change is streamed immediately into the data pipeline, moving from the production app database to a central "landing zone". Real-time ingestion is essential for use cases like fraud detection and live delivery tracking, where a delay of even a few minutes could be costly.

Stage 3: Storage

Storage is not just a passive "disk". It is the backbone that supports every other stage of the lifecycle. Data must be held in systems that are reliable, scalable, and support fast retrieval. Two common types are:

- **Object storage:** for large volumes of unstructured or semi-structured data (e.g. a transaction stored as a raw JSON file in an Amazon S3 bucket)
- **Cloud data warehouses:** for structured data optimised for analytics and querying

Storage choices must balance durability (data is never silently lost), access speed, and compliance requirements.

Stage 4: Transformation (Processing)

This is where data engineers apply business logic to raw data. Typical tasks include removing duplicates, handling missing values, standardising formats, and applying business rules, for instance converting currency for an international transfer, or flagging failed transactions so they don't appear in profit reports. This stage converts technically-correct-but-meaningless raw data into information that is genuinely usable by the business.

Stage 5: Serving (Analysis)

Once transformed, data is delivered to "downstream stakeholders": dashboards, analysts, and machine learning systems. For example, an AI fraud model can analyse a bank transfer in real time to flag it as suspicious, or a recommendation engine can suggest food items based on a customer's order history. This stage is where data finally creates business value.

Stage 6: Archival

Not all data needs to stay in fast, expensive storage forever. After a defined retention period (e.g. five years for a bank), older records are moved to "cold storage" (such as Amazon S3 Glacier) to comply with regulatory and legal requirements, while freeing up capacity in the primary systems.

1.4 Core Dimensions of Data Quality

Before any data can be trusted for decision-making, it must be evaluated against four core quality dimensions:

Dimension	Meaning	Example of Failure
Accuracy	Data should reflect real-world truth	A recorded age of -5, or a bank balance shown as ₹10,000 instead of the correct ₹1,000
Completeness	All required data should be present	A missing age, phone number, or transaction detail

Consistency	The same data should match everywhere it appears	One system shows "Ravi", another shows "Ravi Kumar"; two systems show different balances for the same account
Reliability	Data should be stable and dependable over time	The same query returns different results at different times

1.5 Data Validation and Cleansing

Validation is how we find errors in data. It checks whether data follows defined rules, such as: age must be greater than 0, marks must fall between 0 and 100, or an email must follow a proper format. Cleansing is the step that comes after validation: once problems are found, they are fixed. For example, a dataset with a missing age and a duplicate record is corrected by imputing the missing value and removing the duplicate. The key relationship to remember is: validation finds problems, cleansing fixes them.

1.6 Roles in Data Engineering

In real systems, no single person handles the entire lifecycle. Different roles specialise in different stages, and they depend heavily on one another:

- **Data Engineer:** collects, cleans, and stores data (e.g. handling transaction data in a banking system); they prepare data for use by others
- **Data Analyst:** studies the prepared data and creates reports (e.g. monthly sales or customer analysis); they explain what the data shows
- **Machine Learning Engineer:** builds predictive models using the cleaned data (e.g. fraud detection or recommendation systems); they use data for predictions

A fraud detection system illustrates how these roles connect: the Data Engineer collects and cleans transaction data, the Data Analyst studies patterns in it, and the ML Engineer builds the model that flags suspicious activity. The critical takeaway is that if the data is wrong at the very first stage, every downstream result, including reports, predictions, and decisions, will also be wrong. This is why data quality is

described as a "silent killer": an error doesn't announce itself, it just quietly corrupts everything built on top of it.

1.7 Case Study: A Bank Transfer, End to End

Consider moving ₹10,000 from Account A to Account B, traced through the full lifecycle:

- **Generation:** Customer initiates the transfer via the mobile app; it is recorded in the OLTP database.
- **Ingestion:** Continuous CDC streams the change event from the database logs to a real-time pipeline.
- **Storage:** The ingested data is stored in a cloud data lake/warehouse across hot (fast access) and cold (compliance) tiers.
- **Transformation:** Raw data is cleaned, business rules are applied, and it is enriched with customer data.
- **Serving:** Transformed data feeds BI dashboards, fraud detection systems, and CRM updates.

Possible Failure Points in this Scenario

- **Ingestion failure:** if the CDC tool breaks, the bank won't see fraudulent transfers until it's too late
- **Incorrect processing:** a bug that doubles a transaction value produces a completely wrong income report for the CEO
- **Security breach:** if a storage bucket is left "public", every customer's balance is exposed to the internet

1.8 Data Flow: Connecting the Lifecycle

Data Flow refers to the movement and "velocity" of data through a system. Earlier systems relied mainly on batch processing; increasingly, organisations use real-time streaming. Data Flow can be thought of as the "bloodstream" of the lifecycle. Without it, the stages of Generation, Ingestion, Storage, Transformation, and Serving would simply be disconnected islands with no way of moving information between them.

1.9 Unit 1 Summary

- Data quality is the foundation of every reliable analytics or business system
- The lifecycle, covering Generation, Ingestion, Storage, Transformation, Serving, and Archival, describes how raw data becomes usable information
- Validation finds problems in data; cleansing fixes them
- Data Engineers, Data Analysts, and ML Engineers depend on each other, and a failure at any stage propagates downstream
- Data Flow (batch vs real-time) is what connects the stages of the lifecycle together

Unit 2: Data Processing Using Python

Author: Adrino Rosario James

2.1 Introduction to Data Processing

Organisations generate massive volumes of data from websites, apps, sensors, banking and healthcare systems, and surveys. However, raw data collected from these sources is rarely analysis-ready. Consider a simple student feedback survey:

Student	Rating	Time Spent
Arun	5	2 Hours
Priya	Missing	3 Hours
Arun	5	2 Hours
Kiran	10	1 Hour

This tiny table already shows four classic problems: a missing value (Priya's rating), a duplicate record (Arun appears twice), an invalid value (Kiran's rating of 10 on what should be a 1 to 5 scale), and potential inconsistency in formatting. If analysed directly, results drawn from this data would be inaccurate and misleading.

Data Processing is the collection of activities performed on raw data to transform it into meaningful, reliable, and useful information. It broadly involves: collecting, organising, cleaning, transforming, validating, and integrating data, before finally preparing it for analysis. The final output is called a processed or curated dataset.

2.2 The Data Processing Workflow

A typical workflow moves through the following stages, each of which improves the quality and usefulness of the data:

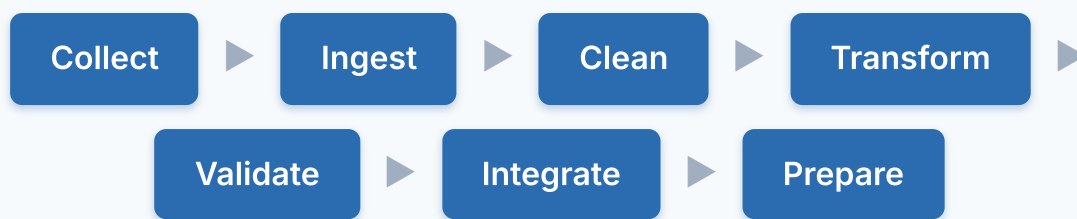


Figure 2.1: The Data Processing Workflow

2.3 Structured Dataset Handling

Structured data is organised in a predefined format of rows and columns, for example, a table of student records where each row is a record and each column is an attribute (Student ID, Name, Marks). Structured data is preferred by most business systems because it is easy to store, query, and analyse: it offers easy storage, fast retrieval, efficient querying, and better analysis compared to unstructured formats.

In Python, structured data is primarily handled using the Pandas DataFrame, which behaves like a digital spreadsheet. A DataFrame provides built-in facilities for reading data, updating records, filtering information, performing calculations, and generating summaries, making it the core tool for almost every data processing task in this unit.

2.4 Dataset Ingestion from Structured Sources

Ingestion is the process of importing data from external sources into a processing environment; without it, data remains locked away and cannot be cleaned or analysed. Common structured sources include:

- **CSV Files:** comma separated values, the most common format in analytics
- **Excel Files:** widely used across organisations for reporting and record-keeping
- **Databases:** systems such as MySQL, PostgreSQL, Oracle, and SQL Server, which large organisations typically use for relational storage

Ingestion is the entry point of the entire pipeline: without it, data cannot be processed, cleaned, or analysed at all.

2.5 Tabular Data Representation

Tabular representation organises information into rows and columns, for example, a registration number, rating, and time-spent table. This format offers three key advantages: readability (humans can understand it easily), efficient querying (specific records can be retrieved quickly), and simplified processing (mathematical and statistical operations become straightforward). Most data processing systems internally convert information into tabular structures before performing any computation.

2.6 Data Quality Assessment

Before cleaning begins, data quality must be assessed to identify potential problems. The common issues to check for are:

- **Missing values:** for example a rating recorded as Null
- **Duplicate records:** the same row appearing more than once
- **Invalid values:** for example a rating of 8 when the valid scale is 1 to 5
- **Inconsistent data:** for example "data cleaning", "Data Cleaning", and "DATA CLEANING" all representing the same category

Identifying these issues before analysis begins is what allows a data engineer to plan the correct cleaning strategy.

2.7 Data Cleaning

Data cleaning is the process of correcting, removing, or replacing problematic data, and it is often the single most time-consuming activity in any data analytics project. The main techniques for handling missing values are:

Method	How It Works	Advantage	Disadvantage
Deletion	Remove records with missing values	Simple; suitable when only a few records are affected	Loss of information
Mean Imputation	Replace missing value with the column average	Preserves dataset size	Can distort the distribution
Median Imputation	Replace missing value with the middle value	Useful when outliers exist	Ignores relationships between variables
Mode Imputation	Replace with the most frequent value	Suitable for categorical data	Can over-represent the majority class
Constant Value Replacement	Replace with a fixed label (e.g. "Unknown")	Frequently used in surveys	Doesn't reflect a real value
Forward Fill	Missing value takes the previous available value	Useful for time-ordered data	Assumes no change occurred
Backward Fill	Missing value takes the next available value	Useful for time-ordered data	Assumes future value applies retroactively

Example (Mean Imputation): given marks 80, 90, unknown, 100, the mean of the known values $(80+90+100)/3 = 90$, so the missing value becomes 90.

2.8 Data Validation

Data validation ensures that data satisfies predefined rules before it is trusted for use. Typical rules include: age cannot be negative, a rating must fall between 1 and 5, and registration numbers must follow the institution's defined format. The benefits of validation are improved accuracy, reduced errors, and better decision-making downstream.

2.9 Data Consistency Checks

Consistency ensures identical information is represented uniformly across a dataset, for example, "Male", "male", and "MALE" should all be standardised into a single format. Strong consistency directly improves reporting accuracy, aggregation quality, and search efficiency.

2.10 Dataset Formatting for Processing Workflows

Raw datasets often contain long column names, mixed data types, and unstructured responses. Formatting tasks that address this include renaming columns, converting categorical values into numerical form, standardising date formats, and removing extra spaces, all of which make a dataset easier to work with programmatically.

2.11 Data Transformation and Preparation

Transformation converts data into forms suitable for analysis, bridging the gap between raw data and analytical, decision-ready data. Key transformation techniques include:

- **Filtering:** extracting records that satisfy specific conditions (e.g. students scoring above 80, employees with more than 5 years of experience)
- **Aggregation:** summarising detailed records into meaningful figures (e.g. average marks, total sales, number of students per department)
- **Integration and Merging:** combining related datasets using a common attribute, such as joining a Student dataset with a Feedback dataset on Registration Number

- **Reshaping:** changing a dataset's structure while preserving its information, through operations such as pivoting, melting, stacking, and unstacking

2.12 Data Normalization and Standardization

Normalization scales values into a common range (commonly 0 to 1) so that features with naturally large values, like income, do not dominate features with naturally small values, like a 1 to 5 rating, purely because of scale. Standardization goes a step further, transforming data so that it has a mean of 0 and a standard deviation of 1. Both techniques eliminate scale differences and are widely used to improve the performance of machine learning models, statistical modelling, and predictive analytics.

2.13 Preparing Curated Datasets

A curated dataset is the final, analysis-ready product of this entire workflow: cleaned, validated, transformed, and properly structured, with no duplicates and no invalid values. Curated datasets are what power Business Intelligence tools, Machine Learning pipelines, Data Warehousing, and Reporting Systems in real organisations.

2.14 Unit 2 Summary

Data processing is a critical stage in the data lifecycle. Raw data collected from various sources often contains errors, inconsistencies, duplicates, and missing values. Through structured dataset handling, ingestion, cleaning, validation, transformation, integration, normalization, and standardization, raw data is converted into curated datasets that support reliable analytics and decision-making. Understanding these concepts is essential before implementing them in Python, because Python commands are simply tools that automate these underlying data processing activities.

References

- Joe Reis, Matt Housley, *Fundamentals of Data Engineering*, O'Reilly Media, 2022.
- Martin Kleppmann, *Designing Data-Intensive Applications*, O'Reilly Media, 2017.
- Thomas L. Floyd, *Digital Computer Fundamentals*, Pearson International, 11th Edition, 2015.
- Paul Crickard, *Data Engineering with Python*, Packt Publishing, 2020.